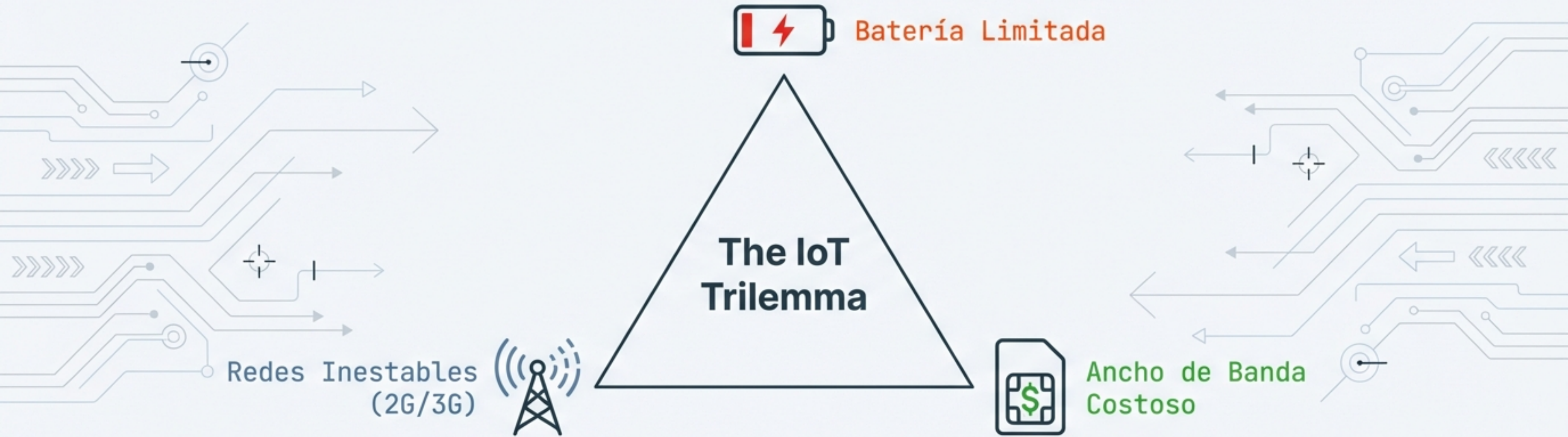




Protocolo SRV-GPS: Mensajería Ligera para IoT

Optimización extrema para entornos de baja
cobertura y batería crítica.

El Desafío de la Ingeniería IoT



Los protocolos web estándar son demasiado pesados.
Necesitamos una **dieta estricta de datos**.

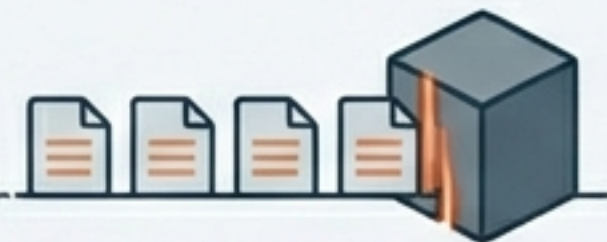
Transporte: La Decisión UDP vs TCP

⚠ TCP - Descartado



Overhead: 20 Bytes
Handshake: Costoso

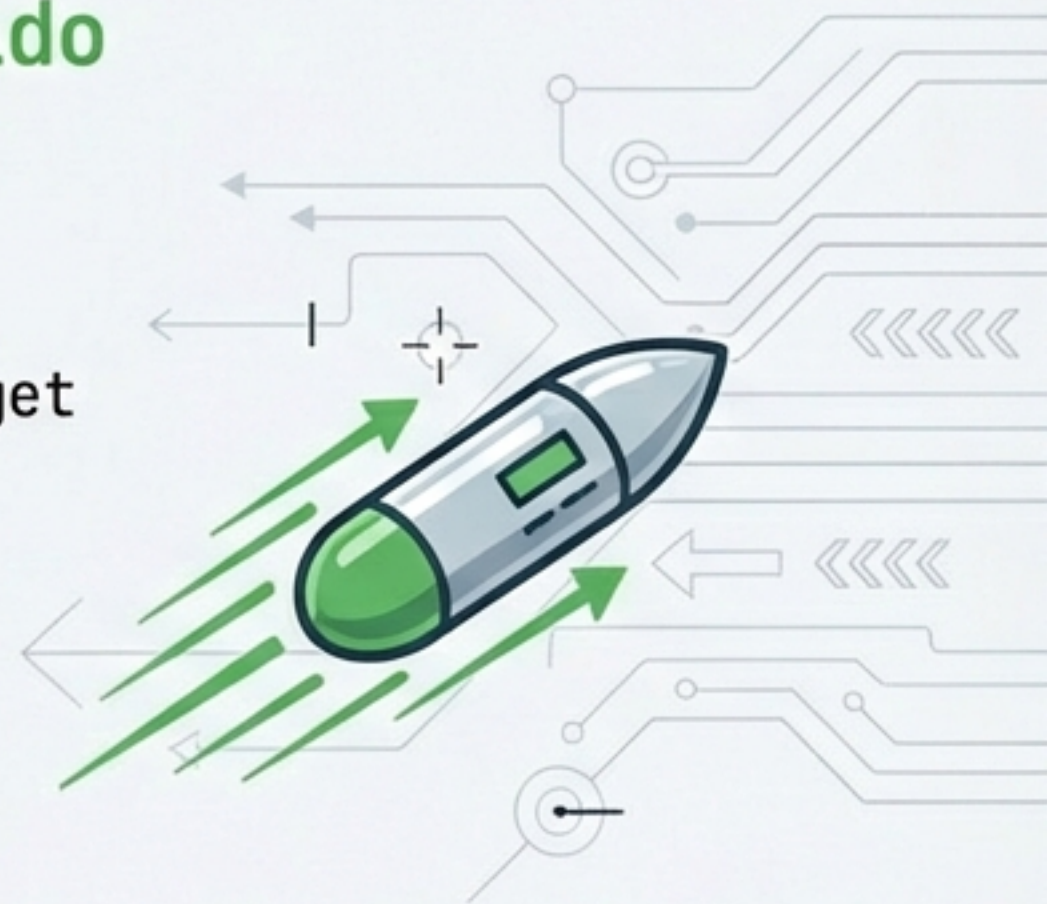
Problema: Head-of-line
blocking (Retrasos)



✅ UDP - Elegido

Overhead: 8 Bytes
Modelo: Fire & Forget

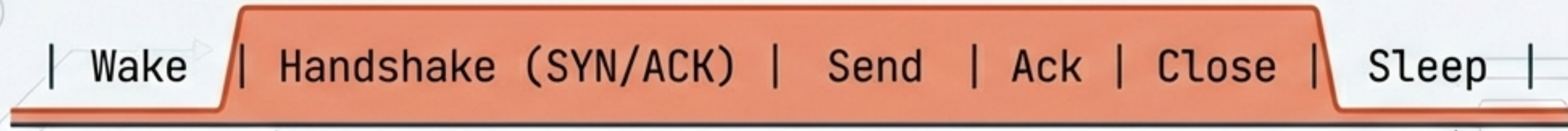
Ventaja: Velocidad
y Eficiencia



Prioridad a la Actualidad: Es mejor perder un paquete antiguo que retrasar el dato actual.

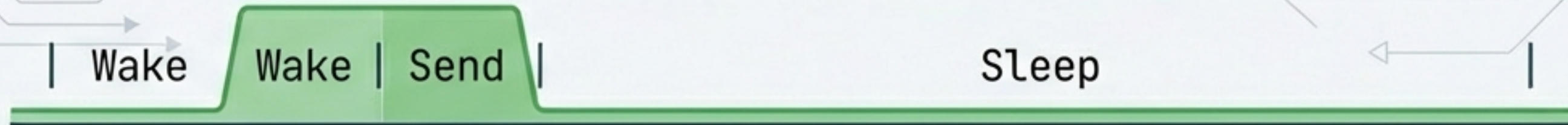
Estrategia de Energía: Deep Sleep

Escenario TCP Standard



↑ Alto Consumo de mAh 

Escenario SRV (UDP)



↓ Ahorro Crítico de Energía 

Reducción drástica del consumo de miliamperios-hora (mAh) al eliminar tiempos de negociación de conexión.

Serialización: Eliminando la Grasa Digital

Comparativa de Formatos de Datos Antes y Después

JSON Standard (~140 Bytes)

```
{  
  "lat": -16.500,  
  "lon": -68.150,  
  "bat": 85,  
  "ts": 17065432  
}
```

grasa digital

grasa digital

Protocolo SRV (~55 Bytes)

SRV | -16.500 | -68.150 | 85 | 17065432

**Reducción del 50% en el tamaño del mensaje
= Ahorro directo en planes de datos.**

Menos bytes, mayor eficiencia de red y energía.

La optimización del protocolo de serialización es clave para minimizar el consumo de ancho de banda y extender la vida de la batería.

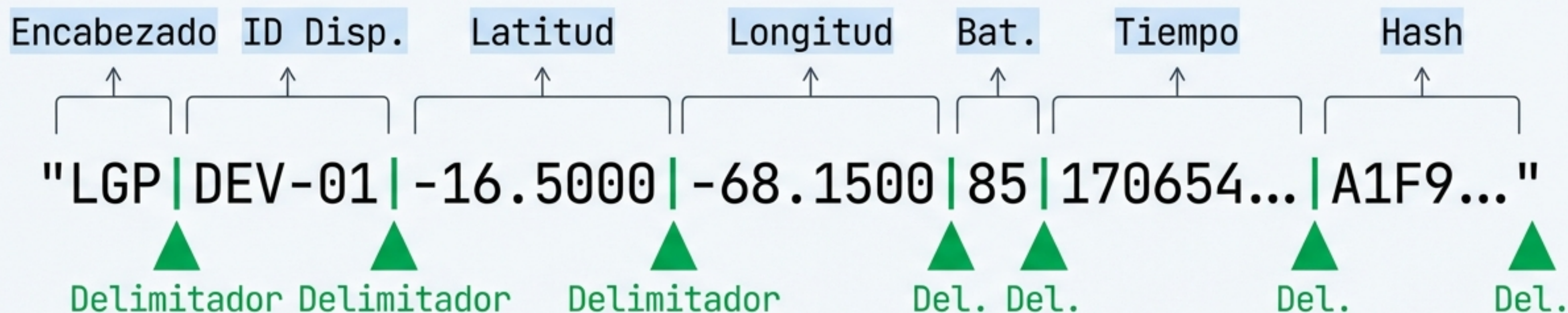
Anatomía de la Trama SRV



Estructura de vagones de tren separados por tuberías (|).

- **HEADER**: Identificador del protocolo
- **ID**: Identidad del dispositivo (Variable)
- **LAT/LON**: Coordenadas geográficas
- **BAT**: Estado de batería (Int)
- **TIMESTAMP**: Marca de tiempo Unix
- **CHECKSUM**: Integridad (Hex)

Legibilidad Humana y Depuración



A pesar de la eficiencia, mantenemos codificación ASCII.

- Permite inspección directa en logs y consola sin herramientas propietarias de decodificación. Facilita el desarrollo ágil.

Seguridad sin el Costo de SSL



Integridad

Hashing SHA-256 (Campo Checksum) para detectar corrupción por ruido en ruido en la red.



Confidencialidad

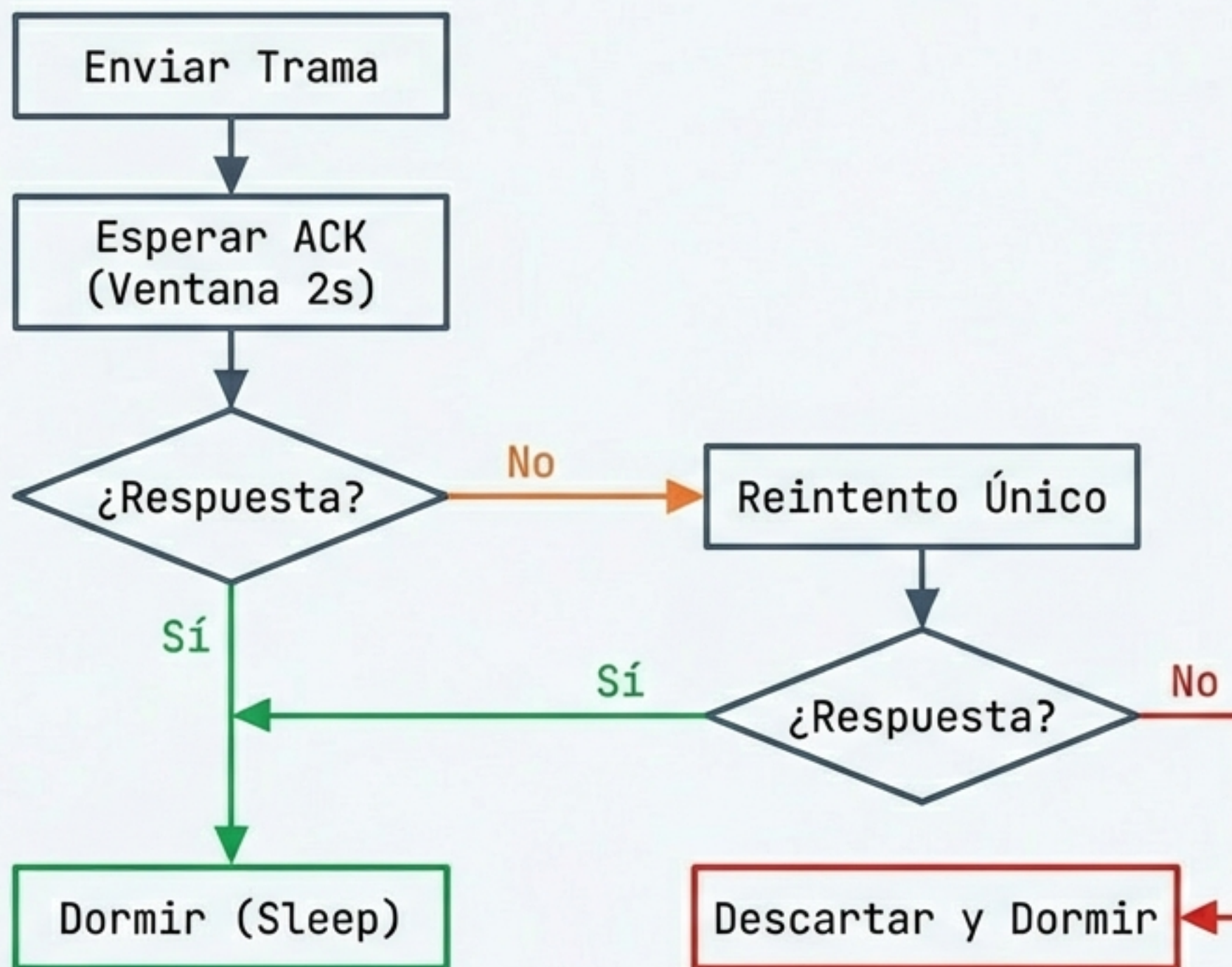
Ofuscación ligera del payload antes de la transmisión.



Autenticación

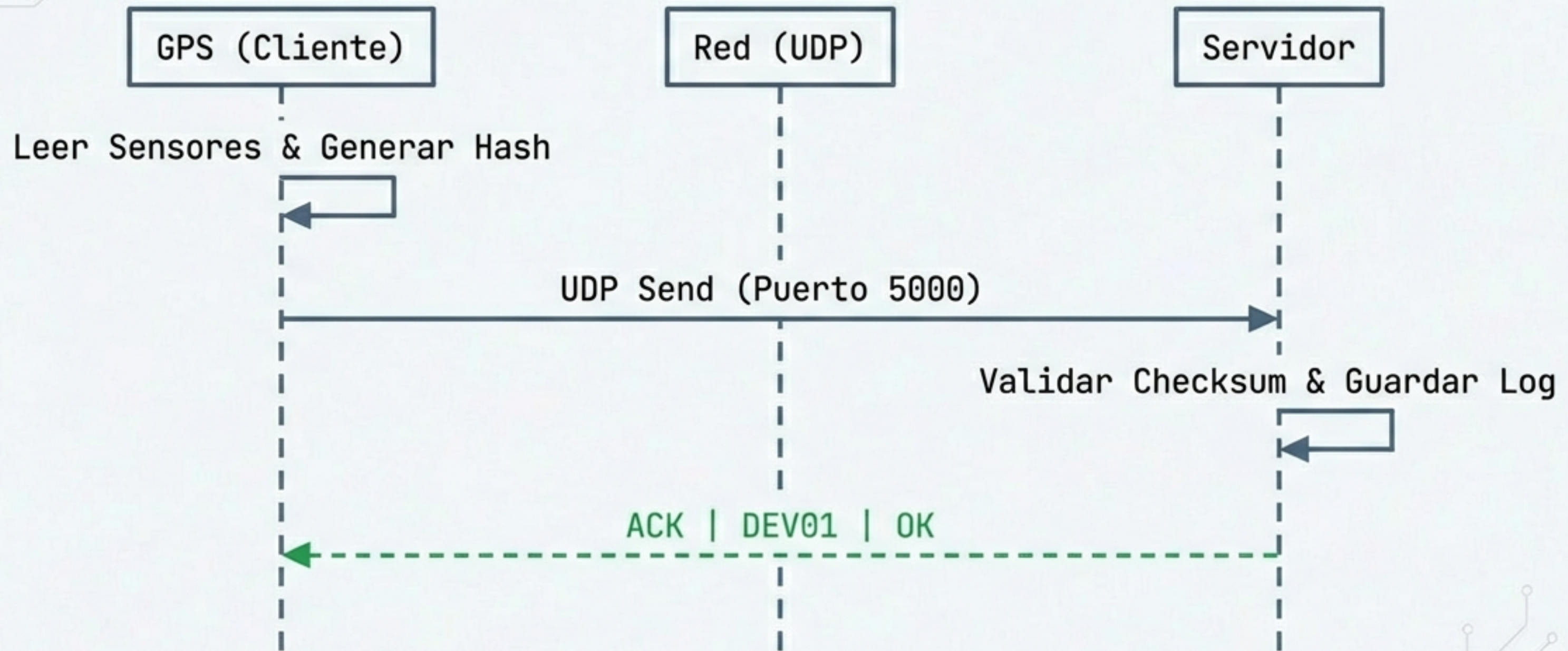
Validación de ID de dispositivo mediante Listas de Control de Acceso (ACL).

Fiabilidad en Capa de Aplicación



Evitamos los 'Bucles de la Muerte' (reintentos infinitos) que drenan la batería.

Arquitectura de Comunicación



Simulación y Diagnóstico en Tiempo Real

```
[OK] Hash Correcto -  
Datos Guardados:  
registro_gps.txt
```

```
[ERROR] Hash Inválido -  
Paquete Descartado  
(Integridad Comprometida)
```

```
[TIMEOUT] Sin respuesta  
del servidor...  
Reintentando (Intento  
1/1)
```


Implementación del Proyecto

Protocolo_SRV/

		servidor.py	Escucha en puerto 5000, valida hashes, gestiona logs.
		cliente.py	Simula movimiento GPS, descarga de batería y estados de sueño.
		registro_gps.txt	Persistencia ligera en archivos planos para auditoría.

Resumen de Ventajas Competitivas



50% Menos Datos que JSON



Cero Latencia de Conexión
(UDP)



Enfoque 'Battery First'
(Deep Sleep)



Resiliencia ante redes
2G/3G inestables



Universidad Mayor de San Simón
Carrera: Ingeniería Informática y Sistemas

Equipo de Desarrollo

| Ramos Gabriel Boris Javier |

| Soliz Alcocer Leandro Wilson |

| Viamont Rico Alvaro |

Fecha: 30 de Enero de 2025

Código y Documentación



| github.com/AlvaroViamont/Protocolo_SRV |

Simplicidad es Eficiencia.